



Git & GitHub

FUNDAMENTALS

Recitation 0.08



bash — your first repo

```
$ git init
```

```
Initialized empty Git repository
```

```
$ git add .
```

```
$ git commit -m "first commit"
```

```
[main (root-commit) a1b2c3d] first commit
```

```
$ git push origin main
```

What you'll learn



01

Version Control

Why VCS exists & the 3 models



02

Meet Git

Snapshots, and Git vs GitHub



03

Setup & Install

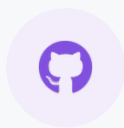
Git, config, SSH/HTTPS, accounts



04

Git on the CLI

init · add · commit · history ·
undo



05

GitHub Basics

Repos, remotes, push & pull



06

Branch & Merge

Branches, conflicts, rebase



07

Collaboration

Forks, pull requests, reviews



08

Best Practices

Cheat sheet, habits

Life without version control

Sound familiar?



report.docx



report_v2.docx



report_v2_FINAL.docx



report_FINAL_final.docx



report_USE_THIS_ONE.docx

...and you still can't find the right one.

The real problems



No history

You can't see what changed, when, or why



No safety net

One bad edit and the old version is gone



No teamwork

Emailing zip files and overwriting each other

What is version control?

A version control system (VCS) records every change to your files over time, **so you can review history, compare versions, and roll back to any point.**



Track history

Every change is logged with who, what & when



Undo anything

Roll back a file or the whole project



Work in parallel

Try ideas on branches without breaking main



Collaborate

Many people, one project, no overwrites

Three models of version control



Local

Versions saved only on your own machine.

Simple, but no sharing and one point of failure.



Centralized

One central server holds the project (e.g. SVN).

Team can share but the server is a bottleneck.



Distributed

Everyone has a full copy of the whole history.

Work offline, no single failure point

Why distributed won

Because every developer clones the entire repository, Git gives you:



Speed



Work offline



Resilience



Cheap branches



Easy sharing



Industry standard

What is Git?



The distributed version control system

Created in 2005 by Linus Torvalds

Built to manage development of the Linux kernel

Free & open source

Fast and reliable

Git ≠ GitHub

Git is the tool that runs on your computer.
GitHub is a website that hosts Git repositories online.



check your version

```
$ git --version  
git version 2.45.0
```

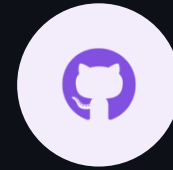
Git vs GitHub



GIT

The tool · Runs on your computer

- Version control software
- Works fully offline
- Created 2005, open source
- Tracks commits & branches
- Used via CLI or an IDE



GITHUB

The platform · A website built around Git

- Hosts your repos in the cloud
- Adds pull requests & reviews
- Issues, wikis, CI/CD, pages
- Founded 2008 (owned by Microsoft)
- Alternatives: GitLab, Bitbucket

How Git thinks: snapshots, not differences

Each commit is a full snapshot of your project at that moment (like a photo!!). Unchanged files are simply linked to the previous snapshot, so it stays fast and tiny.



Installing Git



Windows

Download the installer from git-scm.com, or use a package manager.

```
$ winget install  
Git.Git
```



macOS

Install via Homebrew, or trigger Apple's developer tools.

```
$ brew install git
```



Linux

Use your distro's package manager (Debian / Ubuntu shown).

```
$ sudo apt install git
```

```
git --version
```

First-time configuration

Tell Git who you are. This identity gets stamped on every commit you make.

```
~/ - one-time setup

# your identity (stamped on commits)
$ git config --global user.name "Your Name"
$ git config --global user.email "you@mail.com"
# make 'main' the default branch
$ git config --global init.defaultBranch main
# set default editor
$ git config --global core.editor "code --wait"
$ git config --global -e
# review everything
$ git config --list
```



--system vs --global vs --local

--system sets defaults for every repo of all users;
--global sets defaults for every repo of current user;
--local overrides per-project



Why identity matters

Your name & email label each commit's author in the history



Set it once

You usually configure this a single time per machine

Three ways to use Git

Git is the same engine underneath, these are just different interfaces to it.



Command line

Type git commands in a terminal.

Full power & works everywhere.

Best to learn on



GitHub Desktop

A free point-and-click app.

Friendly for visualizing changes, commits & branches.

Great for beginners



IDE / editor

Built into VS Code, JetBrains, etc.

Commit & push without leaving your code editor.

Convenient

Create your GitHub account

1

Go to github.com

Click Sign up and enter your email

2

Pick a username

This becomes part of your profile & repo URLs

3

Verify your email

Confirm the link GitHub sends you

4

You're in

Free plan includes unlimited public & private repos



What you get, free

- Unlimited repositories
- Collaboration & pull requests
- Issues & project boards
- Free hosting with GitHub Pages

Connecting to GitHub: SSH vs HTTPS

When you push or pull, GitHub must verify it's really you. Two ways to authenticate:



HTTPS

Easiest to start. Uses a Personal Access Token (PAT) instead of your password.



```
https://github.com/you/repo.git
```



Save credentials

```
$ git config --global credential.helper store
```



SSH

Set up a key pair once, then push & pull with no prompts. Great for daily use.



```
git@github.com:you/repo.git
```



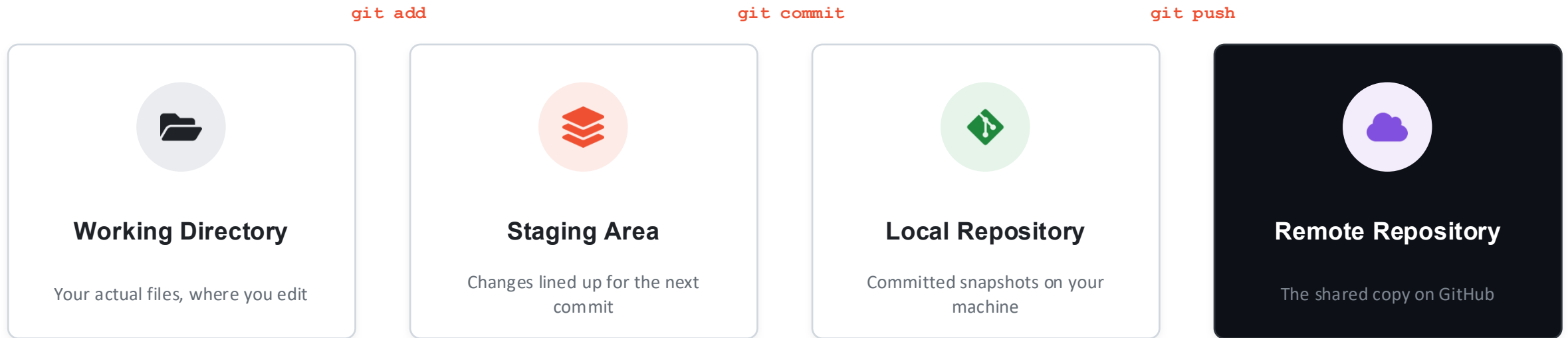
create an SSH key

```
$ ssh-keygen -t ed25519  
$ eval "$(ssh-agent -s)"  
$ ssh-add ~/.ssh/id_ed25519  
$ cat ~/.ssh/id_ed25519.pub
```



The four areas of Git

Every change moves through these places. Knowing them makes every command make sense.



← bring changes back down with `git pull` / `git fetch` / `git clone`

Starting a repo: two methods

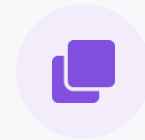


Start fresh

Turn a folder on your computer into a Git repository.

```
new project

$ mkdir my_repo
$ cd my_repo
$ git init
Initialized empty Git repository
```



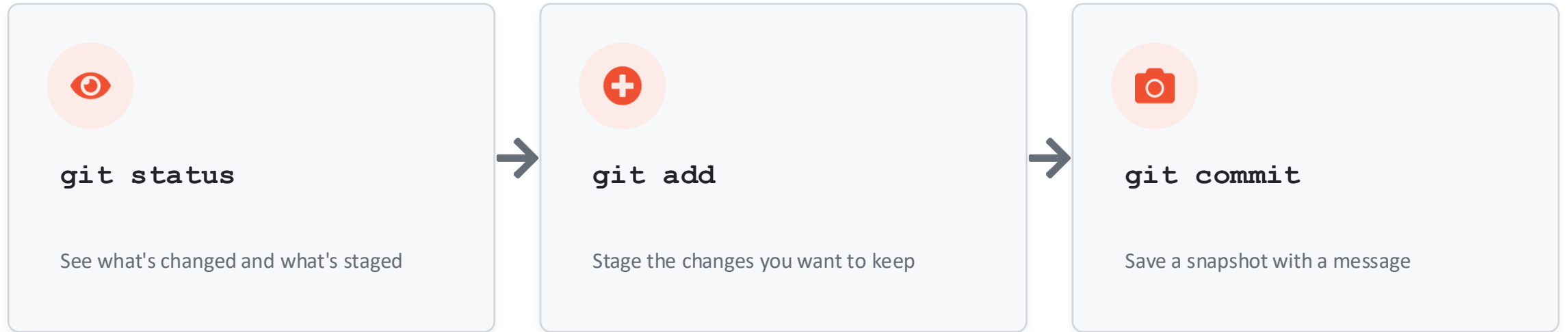
Copy existing

Download a full copy of a repo that already lives on GitHub.

```
clone a remote

$ git clone \
  github.com/you/my-app.git
Cloning into 'my-app'...
done.
```


The everyday loop: `status` → `add` → `commit`



a typical cycle

```
$ git status      # what changed?
$ git add index.html # stage it
$ git commit -m "Add homepage" # snapshot it
```

Your first commit, start to finish

1 Create & enter a folder

2 Initialize the repo

3 Add a file

4 Check the status

5 Stage the change

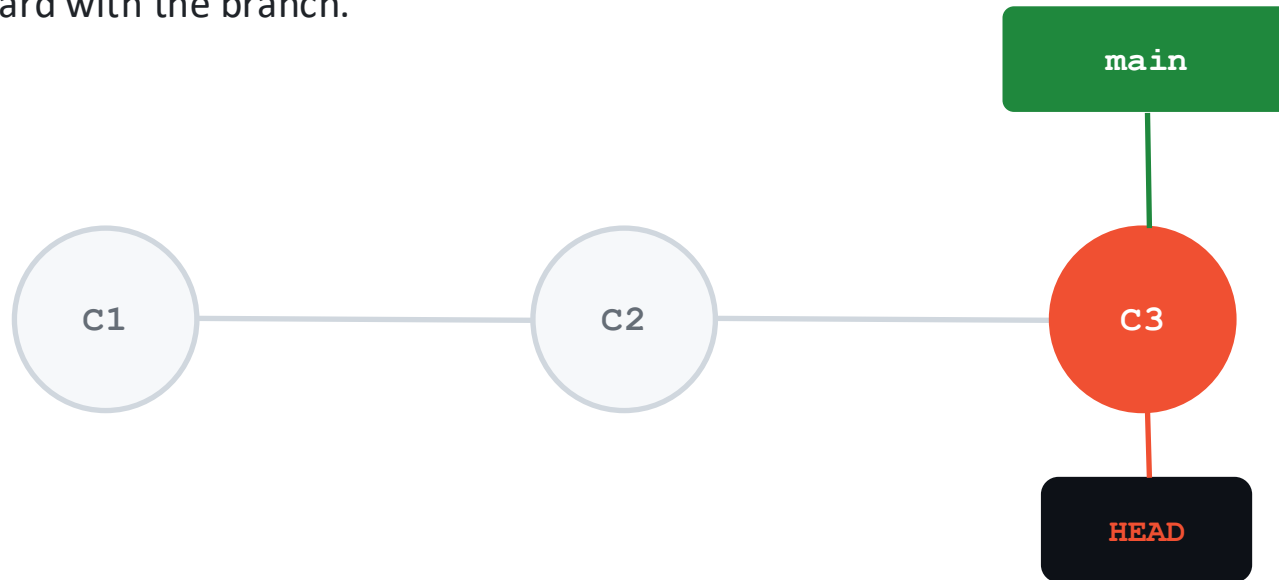
6 Commit the snapshot

```
bash

$ mkdir hello && cd hello
$ git init
$ echo "Hi" > readme.md
$ git status
Untracked files: readme.md
$ git add readme.md
$ git commit -m "Add readme"
[main a1b2c3d] Add readme
1 file changed, 1 insertion(+)
```

HEAD: where you are right now

HEAD is a pointer to your current commit - usually the tip of the branch you have checked out. When you commit, HEAD moves forward with the branch.



Good to know

HEAD~1 means "one commit before HEAD."

Detached HEAD = you checked out a specific commit instead of a branch.

Viewing what happened

```
inspecting history

# full history
$ git log
# compact, one line each
$ git log --oneline
a1b2c3d Add readme
9f8e7d6 Initial commit
# inspect a single commit
$ git show a1b2c3d
```



git log

The complete commit history, newest first



git log --oneline

A scannable one-line-per-commit view



git show <id>

Everything that changed in one commit

Undoing mistakes



Unstage a file

```
git restore --staged file
```

Take it back out of the staging area



Discard local edits

```
git restore file
```

Throw away uncommitted changes to a file



Fix the last commit

```
git commit --amend
```

Reword the message or add a forgotten file



Restore an old version

```
git restore -s <id> file
```

Bring a file back from an earlier commit



Discarding uncommitted changes can't be undone. once they're gone, they're gone. Commit early if unsure.

.gitignore - what NOT to track

Some files shouldn't live in version control: secrets, dependencies, build output, OS junk. List patterns in a .gitignore file and Git will skip them.

```
.gitignore

# dependencies
node_modules/

# secrets & environment
.env

# build output
dist/

# OS / editor
.DS_Store
```



Keep secrets out

Never commit passwords, API keys or .env files



Skip the bulky stuff

node_modules and build folders can be regenerated



Only affects untracked files

Already-committed files keep being tracked until removed

What is GitHub, really?

GitHub is a website that hosts your Git repositories in the cloud and wraps them in tools for sharing, reviewing, and collaborating.



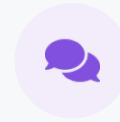
Cloud hosting

A backup of your repo that lives online



Collaboration

Invite others; work together without chaos



Pull requests

Propose, discuss & review changes



Issues

Track bugs, tasks and feature requests



Actions / CI

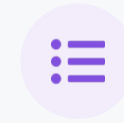
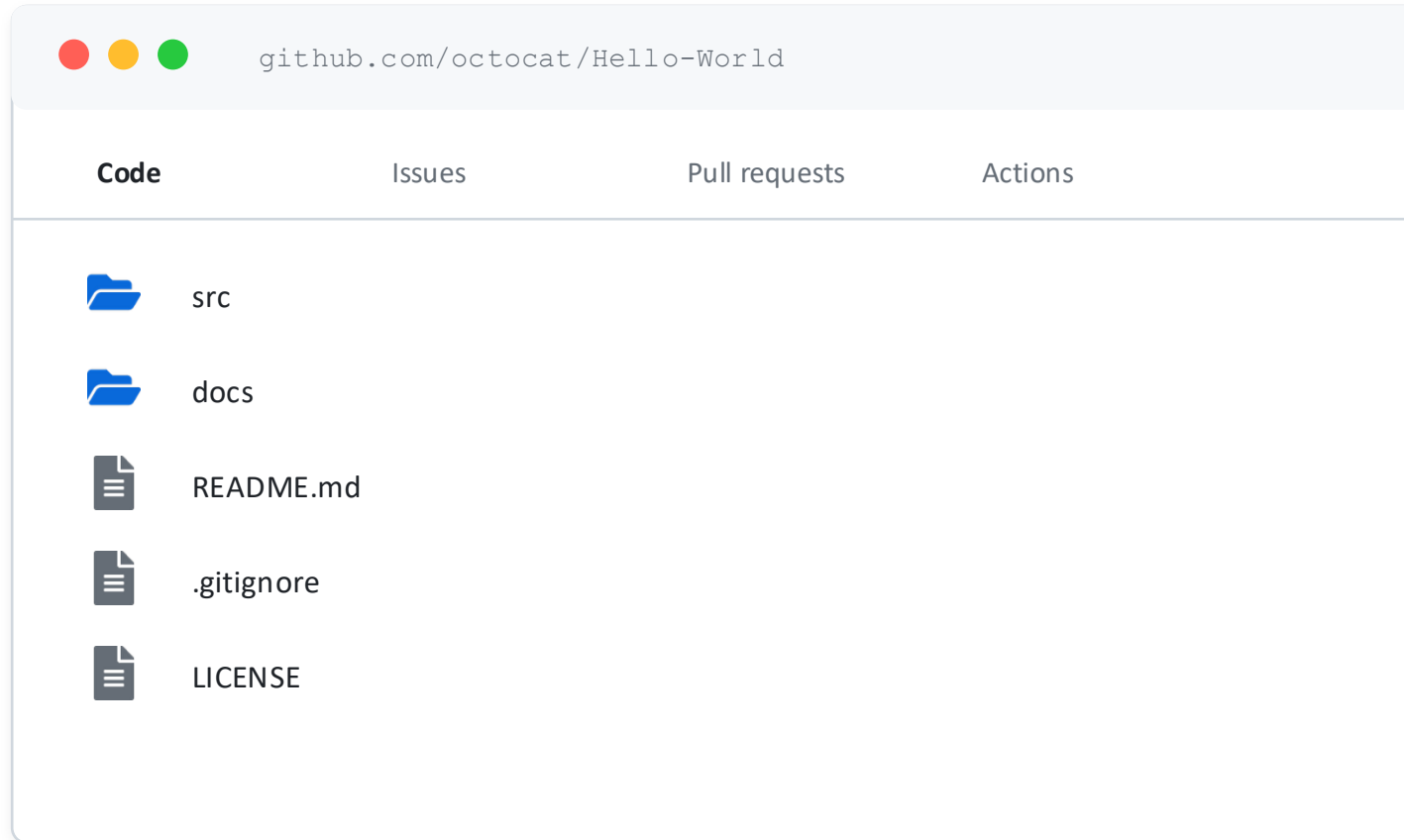
Automate tests & deployments on every push



Pages & wikis

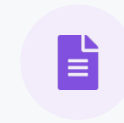
Publish docs and websites straight from a repo

Anatomy of a repository page



Tabs

Code, Issues, Pull requests, Actions & more



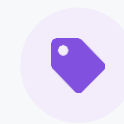
README

The front page of your project, shown automatically



Star / Fork / Watch

Save, copy, or follow a project



Releases & tags

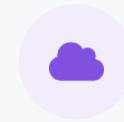
Marked versions you can download

Connecting local → remote

A **remote** is just a nickname for a repo's URL. The default remote is called **origin**.

```
link your repo to GitHub

# add the remote named 'origin'
$ git remote add origin \
  git@github.com:you/my-repo.git
# confirm it
$ git remote -v
origin  git@github.com:you/my-repo.git
```



origin

The conventional name for your main remote



One repo, many remotes

You can add more (e.g. a fork's upstream)



Clone sets it for you

git clone adds origin automatically

Push & pull: keeping in sync



git push

Upload your local commits to the remote, so others (and GitHub) get your work.



```
$ git push origin main
```



git pull

Download commits others have pushed, merging them into your local branch.



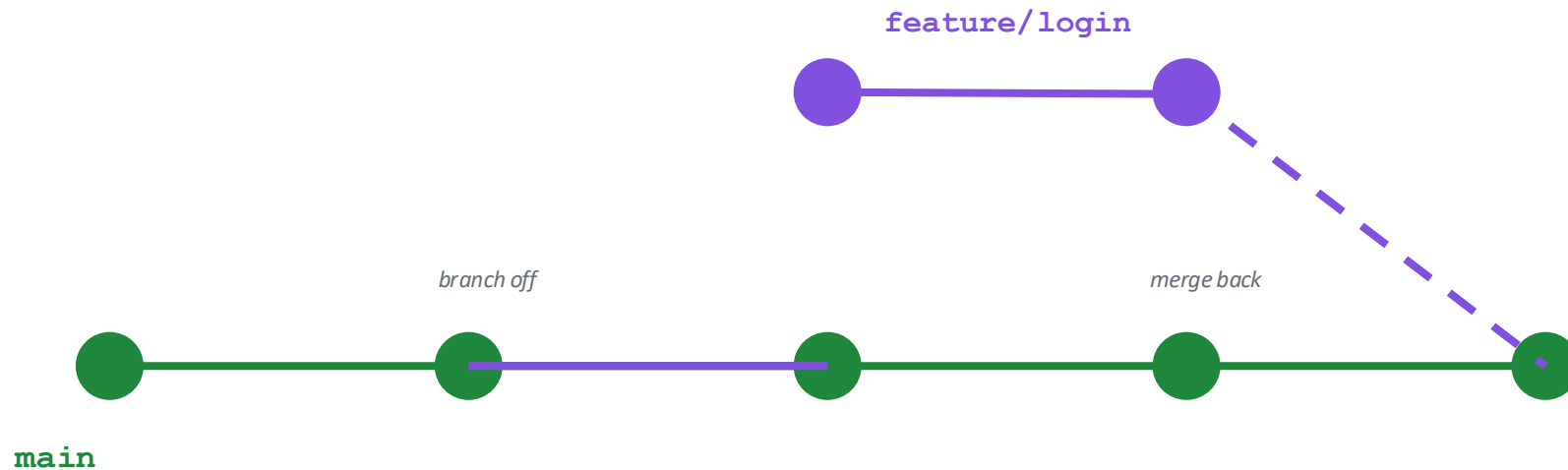
```
$ git pull origin main
```



First push tip: use `git push -u origin main` once. The `-u` or (or `--set-upstream`) links the branches so later you can just type `git push`.

What are branches?

A branch is an independent line of work. Build a feature without touching main, then merge it back when it's ready.



`main` stays stable and shippable while experiments happen safely on their own branches.

Working with branches



branch workflow

```
# create and switch to a new branch
$ git switch -c feature/login
# ...make commits...
$ git switch main
# fold the feature back in
$ git merge feature/login
# tidy up
$ git branch -d feature/login
```

Note: older tutorials use `git checkout -b`, but `git switch` is the newer, clearer equivalent.



git branch

List all branches; * marks the current one



git switch -c name

Create a new branch and switch to it



git switch name

Jump to an existing branch



git merge name

Bring another branch's work into this one

Build a feature on a branch

- 1 Branch off main
- 2 Make your changes
- 3 Commit on the branch
- 4 Switch back to main
- 5 Merge the branch in
- 6 Delete the merged branch

```
bash

$ git switch -c feature/nav
Switched to a new branch 'feature/nav'

$ echo "nav" >> app.py

$ git commit -am "Add nav"

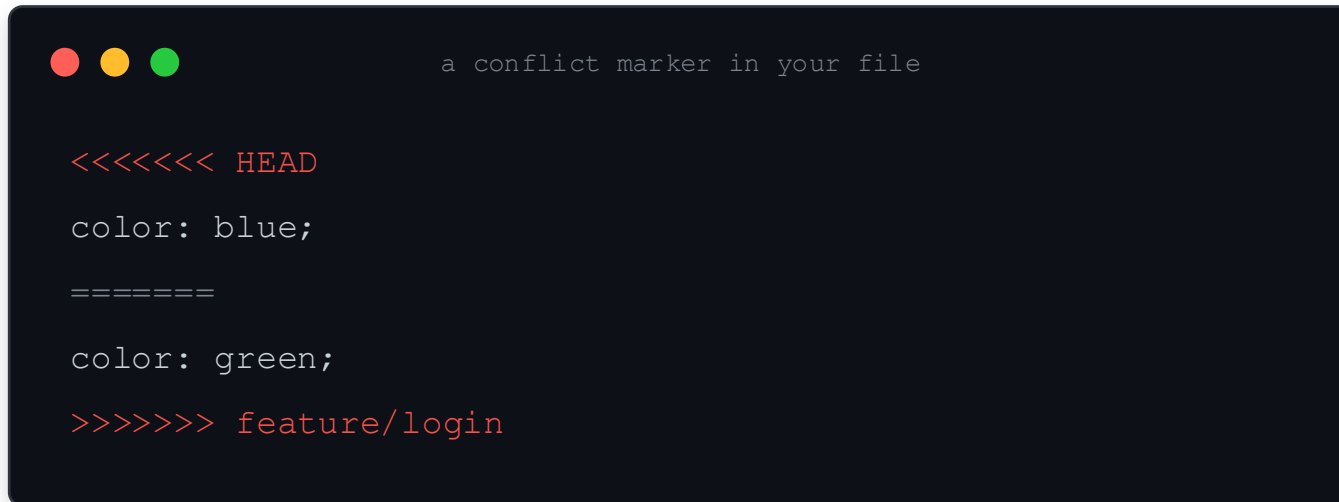
$ git switch main

$ git merge feature/nav
Updating 9f8e7d6..a1b2c3d Fast-forward

$ git branch -d feature/nav
```

Merge conflicts !!!

A conflict happens when two branches change the same lines. Git pauses and asks you to choose. It's normal, not an error.



```
a conflict marker in your file

<<<<<< HEAD
color: blue;
=====
color: green;
>>>>>> feature/login
```



1 · Open the file

Find the <<<<, ==, >>>> markers



2 · Pick the result

Edit to the version you want; delete the markers



3 · Stage & commit

git add the file, then commit to finish the merge

Merge vs rebase (the idea)



Merge

Joins two branches with a new "merge commit." History keeps its real, branching shape.

✓ Safe & honest ✗ Can look messy



Rebase

Replays your commits on top of another branch. History becomes a clean, straight line.

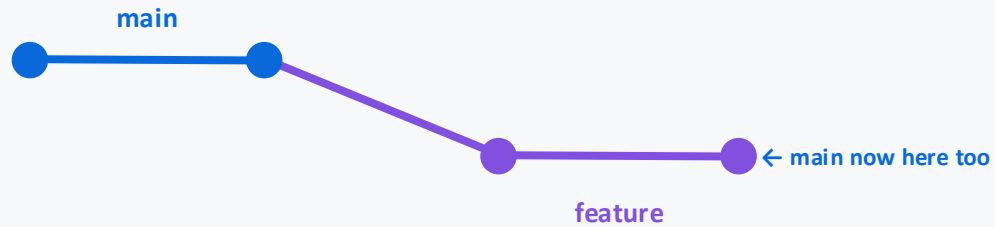
✓ Tidy history ✗ Rewrites commits



The golden rule: never rebase commits you've already pushed and shared. Rewriting public history breaks everyone else's copy.

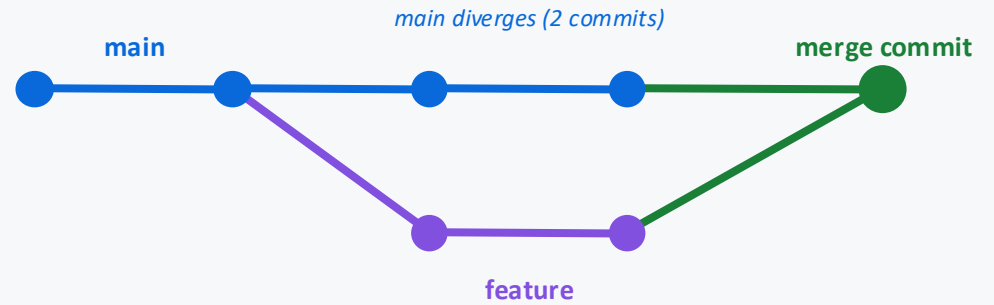
Four ways to combine branches

Fast-forward merge



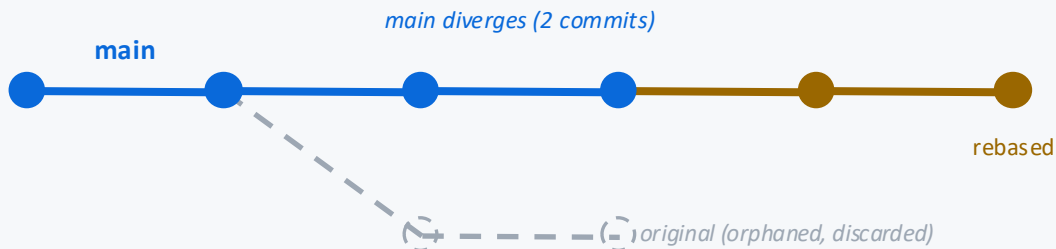
Main has no commits past the branch point, so the fast-forward just moves main's pointer to feature's tip; both commits become main's history, and no new commit is created.

Merge (three-way)



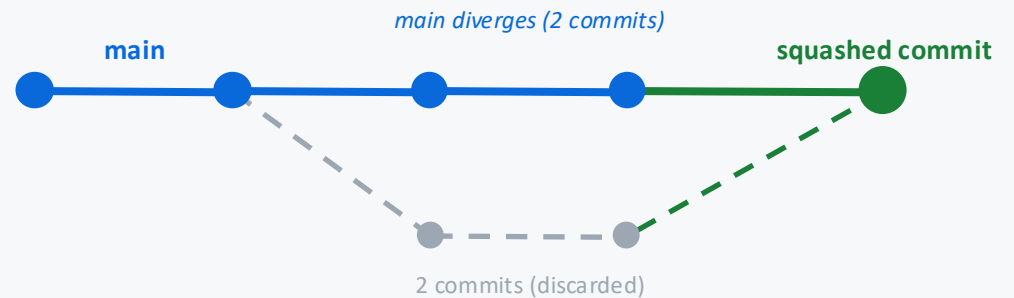
Both branches advanced independently — a new commit ties them together.

Rebase



Same starting point as a merge, the feature is replayed onto main's new tip instead.

Squash merge

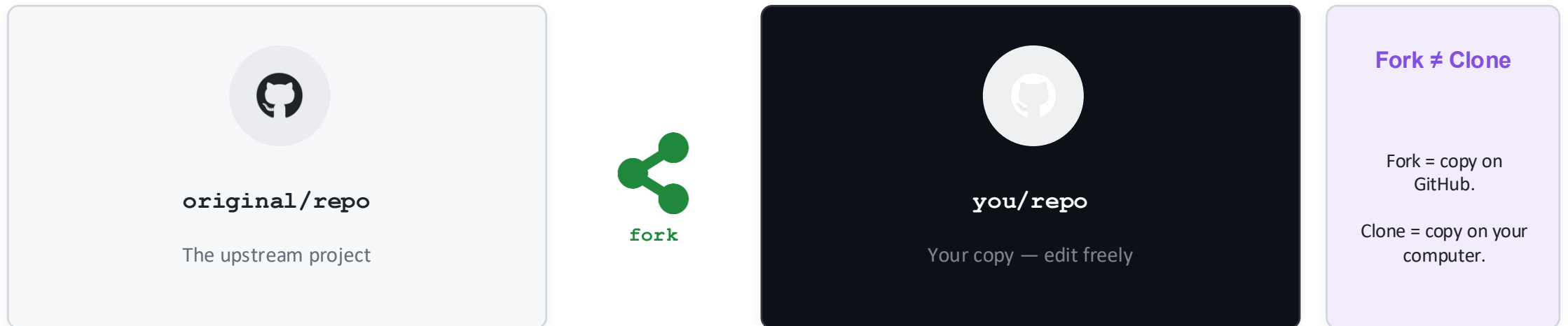


Feature's commits are combined into one new commit after main's own tip.

● Main ● Feature ● New / merge commit ○ Discarded / not-yet-created

Forking a repository

A fork is your own server-side copy of someone else's repo, living under your GitHub account. It's how you contribute when you don't have write access.



Pull requests (the heart of GitHub)

A pull request (PR) proposes merging one branch into another, wrapped in discussion, review, and automated checks.

Add login page #42

Open

you wants to merge `feature/login` into `main`



3 commits · 2 files changed



2 reviewers · 1 comment



All checks have passed



Review before merge

Teammates read your changes and suggest fixes



Discussion in context

Comment on exact lines of code



Automated checks

Tests & CI run before anything is merged

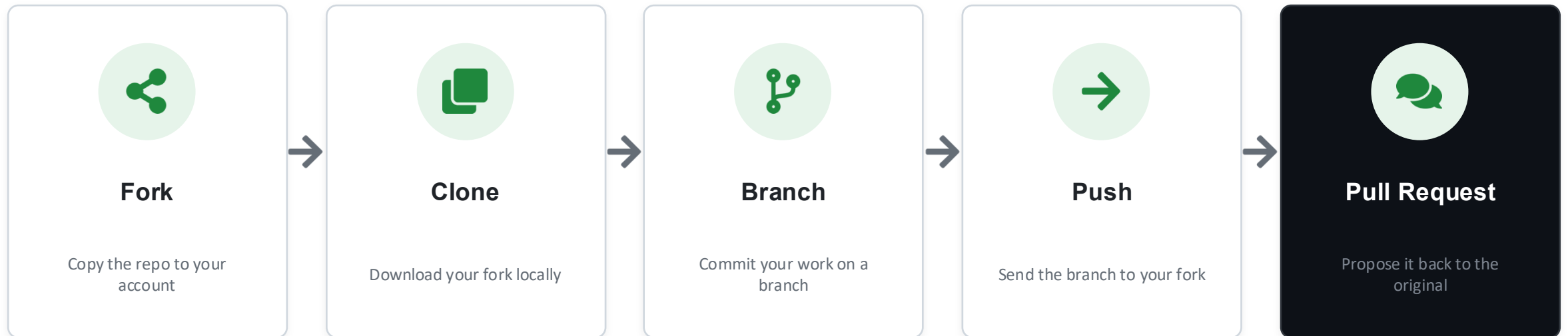


A permanent record

Why a change was made stays searchable forever

The GitHub collaboration model

The classic open-source flow - a fork, a branch, a push, a pull request.



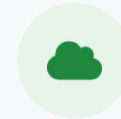
On a team where you have write access, you skip the fork : just branch, push, and open a PR directly.

Keeping your fork in sync

The original repo keeps moving. Add it as a second remote called upstream and pull its updates into your fork.

```
sync with upstream

# point to the original repo (once)
$ git remote add upstream \
  github.com/original/repo.git
# pull its latest main
$ git fetch upstream
$ git switch main
$ git merge upstream/main
```



origin

Your fork - where you push your work



upstream

The original - where you pull updates from



Sync often

Reduces conflicts when your PR is reviewed

Core terminology - your cheat list



Repository

Your project + its entire history (a "repo")



Branch

A movable line of work; default is main



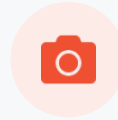
Clone

Download a full copy of a remote repo



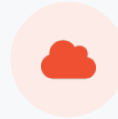
Staging area

The "on-deck" zone for your next commit



Commit

A saved snapshot of changes, with a message



Remote

A repo hosted elsewhere (e.g. on GitHub)



Push / Pull

Send commits up / bring commits down



Fork

Your own copy of someone else's GitHub repo

Command cheat sheet



Setup

<code>git config --global user.name</code>	<i>set your name</i>
<code>git init</code>	<i>start a repo</i>
<code>git clone <url></code>	<i>copy a repo</i>



Save work

<code>git status</code>	<i>what changed</i>
<code>git add <file></code>	<i>stage changes</i>
<code>git commit -m "..."</code>	<i>snapshot</i>
<i>——— Temporarily save uncommitted changes ———</i>	
<code>git stash</code>	<i>save for later</i>
<code>git stash pop</code>	<i>bring it back</i>
<i>—— Undo commits by moving HEAD pointer backwards ——</i>	
<code>git reset --soft HEAD~1</code>	<i>keep changes staged</i>
<code>git reset --hard HEAD~1</code>	<i>discard all changes</i>



Branch & merge

<code>git switch -c <name></code>	<i>new branch</i>
<code>git branch</code>	<i>list of branches</i>
<code>git merge <name></code>	<i>merge in</i>
<code>git log --oneline</code>	<i>history</i>
<code>git diff HEAD~1</code>	<i>compare with prev commit</i>
<code>git diff branch1 branch2</code>	<i>compare two branches</i>
<code>git commit --amend</code>	<i>modify most recent commit</i>
<code>git cherry-pick <commit-hash></code>	<i>apply a commit from other branch</i>



Sync

<code>git push -u origin main</code>	<i>first push</i>
<code>git pull</code>	<i>get updates</i>
<code>git remote -v</code>	<i>list remotes</i>

Habits that make Git easy



Commit small & often

Each commit = one logical change you can describe



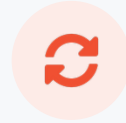
Write clear messages

Imperative & specific: "Fix login redirect"



One branch per task

Keep main clean; isolate every feature or fix



Pull before you push

Stay current to avoid surprise conflicts



Keep PRs small

Small diffs get reviewed faster and merged sooner



Never commit secrets

Keys & passwords belong in .env, not in history

Common mistakes & quick fixes



Forgot to pull first

Fix: `git pull --rebase` to replay your commits on top



Vague commit messages

Fix: Rewrite the latest with `git commit --amend`



Committed a huge file

Fix: Add it to `.gitignore`; use Git LFS for large assets

Look how far you've come



Understood version control & why Git



Installed and configured Git on machine



status → add → commit loop



Connected a local repo to GitHub & pushed code



Created branches, merged them & rebase



Fork and collaborate



You're ready to build.

You now have the fundamentals to version your work and collaborate with anyone, anywhere.

All the Best!!

```
bash

$ git add .
$ git commit -m \
    "End of Recitation"
[main 1a2b3c4] End of Recitation
$ git push
Everything up-to-date ✓
```

<https://git-scm.com/book/en/v2>